

Ejercicios de Funciones en Python

$\{f(x)\}$ Reutiliza, organiza y simplifica tu código

De PSeInt a Python — Guía Práctica Paso a Paso

Docente: Jhonathan Arley Peñaloza Sierra

Skill: Introducción a python

Definir · Invocar · Parámetros · Argumentos · Retorno

Conceptos Clave: Funciones en Python

Antes de comenzar los ejercicios, repásalos cada vez que tengas dudas. Una función es como una receta: tiene un nombre, ingredientes (parámetros), pasos (instrucciones) y un resultado final (retorno).

Anatomía de una función en Python:

```
def nombre_funcion(parametro1, parametro2):    # DEFINIR
    # Instrucciones (cuerpo de la función)
    resultado = parametro1 + parametro2
    return resultado                          # RETORNO

# INVOCAR (llamar) la función
valor = nombre_funcion(5, 3)    # 5 y 3 son ARGUMENTOS
print(valor)                   # Imprime: 8
```

Concepto	Descripción
def	Palabra clave para DEFINIR (crear) una función en Python.
Parámetros	Variables que la función recibe al definirla. Van dentro de los paréntesis en la definición.
Argumentos	Valores concretos que envías al llamar (invocar) la función.
return	Devuelve un resultado al lugar donde se llamó la función. Es opcional.
Invocar	Llamar a la función por su nombre, pasándole argumentos entre paréntesis.
Alcance (scope)	Las variables creadas dentro de una función solo existen ahí dentro.

Referencia: PSeInt vs Python para funciones

PSeInt (Pseudocódigo)	Python
<pre>Funcion resultado <- sumar(a, b) resultado <- a + b Fin Funcion total <- sumar(5, 3)</pre>	<pre>def sumar(a, b): resultado = a + b return resultado total = sumar(5, 3)</pre>

Diferencias clave con PSeInt

En Python usas `def` en lugar de `Funcion` para definir una función.

No se escribe `Fin Funcion`: el bloque termina cuando acaba la indentación.

Usas `return` en vez de `Devolver` para retornar un valor.

Los parámetros NO necesitan tipo (no se escribe `Como Entero`).

La invocación es igual: `nombre(argumento1, argumento2)`.

IMPORTANTE: Debes definir la función ANTES de llamarla en tu código.

PARTE 1: EJERCICIOS GUIADOS

Sigue cada paso como si estuvieras con un tutor a tu lado

Tema 1: Definir e Invocar una Función

Una función es un bloque de código reutilizable con un nombre. Primero la defines (creas la receta) y luego la invocas (usas la receta). Piensa en ella como una máquina: le metes datos (parámetros), los procesa, y te devuelve un resultado (retorno).

Ejercicio 1.1 — Mi primera función: saludar

Enunciado

Crea una función llamada saludar que reciba un nombre como parámetro y muestre en pantalla el mensaje "Hola, [nombre]! Bienvenido a Python.". Luego invócala con tres nombres diferentes.

Guía paso a paso:

Paso 1: Define la función con `def`, un nombre y un parámetro.

```
def saludar(nombre):  
    print(f"Hola, {nombre}! Bienvenido a Python.")
```

`def` es la palabra clave para definir funciones en Python. El parámetro `nombre` es una variable que recibirá el valor que le pasemos al llamarla. Los dos puntos (`:`) al final son obligatorios. Todo lo indentado (con 4 espacios) es el cuerpo de la función. En PSeInt usarías: Funcion saludar(nombre).

Paso 2: Invoca (llama) la función con diferentes argumentos.

```
# Invocar la función 3 veces con diferentes valores  
saludar("Juan")  
saludar("María")  
saludar("Carlos")
```

Cuando escribes `saludar("Juan")`, el valor "Juan" (argumento) se asigna al parámetro `nombre` dentro de la función. Es como decirle a la receta: "usa estos ingredientes". Cada vez que la llamas, se ejecuta todo el código de la función.

Parámetro vs Argumento (no los confundas)

PARÁMETRO = la variable en la DEFINICIÓN: `def saludar(nombre):` ← `nombre` es parámetro.

ARGUMENTO = el valor en la INVOCACIÓN: `saludar("Juan")` ← "Juan" es argumento.

Analogía: el parámetro es el espacio vacío en la receta, el argumento es el ingrediente real.

Comparación PSeInt vs Python:

En PSeInt (Pseudocódigo)	En Python
<pre> Funcion saludar(nombre) Escribir "Hola, ", nombre Fin Funcion saludar("Juan") </pre>	<pre> def saludar(nombre): print(f"Hola, {nombre}!") saludar("Juan") </pre>

Mini Reto 1.1

Modifica la función para que reciba DOS parámetros: nombre y edad.

Que muestre: "Hola, Juan! Tienes 20 años."

Pista: def saludar(nombre, edad):

Solución sugerida — Ejercicio 1.1

```

def saludar(nombre):
    print(f"Hola, {nombre}! Bienvenido a Python.")

# Invocaciones
saludar("Juan")      # Hola, Juan! Bienvenido a Python.
saludar("María")     # Hola, María! Bienvenido a Python.
saludar("Carlos")    # Hola, Carlos! Bienvenido a Python.

# Mini reto: con dos parámetros
def saludar_edad(nombre, edad):
    print(f"Hola, {nombre}! Tienes {edad} años.")

saludar_edad("Juan", 20)

```

Explicación:

Esta función NO usa return porque su trabajo es solo mostrar un mensaje (print).

Las funciones que no retornan nada se llaman funciones sin retorno o procedimientos.

Puedes tener tantos parámetros como necesites, separados por comas.

Ejercicio 1.2 — Función con retorno: sumar dos números

Enunciado

Crea una función llamada `sumar` que reciba dos números como parámetros, calcule la suma y DEVUELVA (retorne) el resultado. Luego invócala y guarda el resultado en una variable.

Guía paso a paso:

Paso 1: Define la función con `return`.

```
def sumar(a, b):
    resultado = a + b
    return resultado
```

`return` devuelve un valor al lugar donde se llamó la función. Es como decir: "aquí está el resultado". Después de un `return`, la función TERMINA (no ejecuta más líneas). En PSeInt usarías: Devolver resultado.

Paso 2: Invoca la función y guarda el resultado.

```
total = sumar(5, 3)
print(f"La suma es: {total}")    # La suma es: 8
```

Cuando escribes `total = sumar(5, 3)`, Python ejecuta la función con `a=5` y `b=3`, calcula `5+3=8`, y el `return` devuelve 8. Ese 8 se guarda en la variable `total`. Sin el `return`, `total` sería `None` (nada).

Paso 3: Puedes usar la función muchas veces con valores diferentes.

```
r1 = sumar(10, 20)    # r1 = 30
r2 = sumar(7, -3)     # r2 = 4
r3 = sumar(100, 200)  # r3 = 300

# Incluso puedes usar el resultado directamente en print:
print(f"Resultado: {sumar(15, 25)}")    # Resultado: 40
```

`print()` vs `return` (diferencia fundamental)

`print()` MUESTRA un mensaje en pantalla, pero no devuelve un valor para usar después.

`return` DEVUELVE un valor que puedes guardar en una variable y usar más adelante.

Ejemplo: si una función usa `print()`, no puedes hacer `total = mi_funcion(5)` para guardar el resultado.

Si necesitas USAR el resultado después, usa `return`. Si solo quieres MOSTRAR algo, usa `print()`.

Mini Reto 1.2

Crea una función llamada `restar` que reciba dos números y retorne la resta.

Crea otra llamada `multiplicar` que retorne la multiplicación.

Usa ambas funciones para calcular: `restar(10, 3) + multiplicar(2, 4)`.

Solución sugerida — Ejercicio 1.2

```
def sumar(a, b):  
    return a + b
```

```
def restar(a, b):  
    return a - b
```

```
def multiplicar(a, b):  
    return a * b
```

```
# Uso
```

```
total = sumar(5, 3)  
print(f"Suma: {total}")
```

```
# Mini reto: combinar funciones
```

```
combinado = restar(10, 3) + multiplicar(2, 4)  
print(f"Resultado combinado: {combinado}") # 7 + 8 = 15
```

Explicación:

Puedes escribir `return a + b` directamente, sin crear la variable resultado.

Las funciones que retornan un valor se pueden usar dentro de expresiones.

Reutilización: escribe la lógica UNA vez, úsala las veces que quieras.

Tema 2: Funciones con Fórmulas Matemáticas

Las funciones son perfectas para encapsular fórmulas que necesitas reutilizar. Defines la fórmula una vez y la aplicas cuantas veces quieras con diferentes valores.

Ejercicio 2.1 — Área de un triángulo

Enunciado

Crea una función llamada `calcular_area_triangulo` que reciba la base y la altura como parámetros, calcule el área con la fórmula ($\text{área} = \text{base} \times \text{altura} / 2$) y retorne el resultado. Calcula el área de tres triángulos diferentes.

Guía paso a paso:

Paso 1: Define la función con la fórmula.

```
def calcular_area_triangulo(base, altura):
    area = (base * altura) / 2
    return area
```

La función recibe dos parámetros (base y altura), aplica la fórmula, y devuelve el resultado con `return`. Nota el nombre descriptivo: `calcular_area_triangulo` dice exactamente qué hace la función.

Paso 2: Llama la función con diferentes valores.

```
area1 = calcular_area_triangulo(5, 8)
area2 = calcular_area_triangulo(3, 6)
area3 = calcular_area_triangulo(10, 4)

print(f"Área 1: {area1}")    # Área 1: 20.0
print(f"Área 2: {area2}")    # Área 2: 9.0
print(f"Área 3: {area3}")    # Área 3: 20.0
```

Escribimos la fórmula UNA sola vez dentro de la función. Sin funciones, tendríamos que repetir $(\text{base} * \text{altura}) / 2$ cada vez. Si la fórmula cambia, solo la corriges en UN lugar.

Comparación PSeInt vs Python:

En PSeInt (Pseudocódigo)	En Python
<pre>Funcion area<-calcular_area(b, a) area <- (b * a) / 2 Fin Funcion r <- calcular_area(5, 8)</pre>	<pre>def calcular_area(b, a): area = (b * a) / 2 return area r = calcular_area(5, 8)</pre>

Mini Reto 2.1

Crea una función `calcular_area_rectangulo(base, altura)` que retorne `base * altura`.
 Crea otra `calcular_area_circulo(radio)` que retorne `3.14159 * radio ** 2`.
 Calcula y muestra el área de un rectángulo de 6x4 y un círculo de radio 5.

Solución sugerida — Ejercicio 2.1

```
def calcular_area_triangulo(base, altura):
    return (base * altura) / 2

def calcular_area_rectangulo(base, altura):
    return base * altura

def calcular_area_circulo(radio):
    return 3.14159 * radio ** 2

# Triángulos
print(f"Área triángulo: {calcular_area_triangulo(5, 8)}")

# Mini reto
print(f"Área rectángulo: {calcular_area_rectangulo(6, 4)}")
print(f"Área círculo: {round(calcular_area_circulo(5), 2)}")
```

Explicación:

Puedes escribir `return` con la expresión directa, sin crear una variable intermedia.
`round()` redondea el resultado del círculo a 2 decimales.
 Buena práctica: nombres de funciones descriptivos que indiquen QUÉ hacen.

Ejercicio 2.2 — Convertir Celsius a Fahrenheit

Enunciado

Crea una función llamada `celsius_a_fahrenheit` que reciba una temperatura en grados Celsius y retorne su equivalente en Fahrenheit usando la fórmula: $^{\circ}\text{F} = (^{\circ}\text{C} \times 9/5) + 32$. Prueba con varias temperaturas.

Guía paso a paso:

Paso 1: Define la función con la fórmula de conversión.

```
def celsius_a_fahrenheit(celsius):
    fahrenheit = (celsius * 9/5) + 32
    return fahrenheit
```

La función recibe UN solo parámetro (`celsius`), aplica la fórmula y retorna el resultado. Esto es exactamente el ejemplo de la diapositiva, pero ahora en Python.

Paso 2: Prueba con varias temperaturas.


```
# Convertir 25°C
temp = celsius_a_fahrenheit(25)
print(f"25°C = {temp}°F")      # 25°C = 77.0°F

# Más conversiones
print(f"0°C = {celsius_a_fahrenheit(0)}°F")      # 32.0°F
print(f"100°C = {celsius_a_fahrenheit(100)}°F")  # 212.0°F
print(f"-40°C = {celsius_a_fahrenheit(-40)}°F")  # -40.0°F
```

Comparación PSeInt vs Python:

En PSeInt (Pseudocódigo)	En Python
<pre>Funcion f <- celsius_a_fahr(c) f <- (c * 9/5) + 32 Fin Funcion t <- celsius_a_fahr(25)</pre>	<pre>def celsius_a_fahr(c): f = (c * 9/5) + 32 return f t = celsius_a_fahr(25)</pre>

Mini Reto 2.2

Crea la función inversa: `fahrenheit_a_celsius(fahrenheit)`.

Fórmula: $^{\circ}\text{C} = (^{\circ}\text{F} - 32) \times 5/9$.

Prueba que sea correcta: `fahrenheit_a_celsius(77)` debería dar 25.

Solución sugerida — Ejercicio 2.2

```
def celsius_a_fahrenheit(celsius):
    return (celsius * 9/5) + 32

def fahrenheit_a_celsius(fahrenheit):
    return (fahrenheit - 32) * 5/9

# Pruebas
print(f"25°C = {celsius_a_fahrenheit(25)}°F")
print(f"77°F = {fahrenheit_a_celsius(77)}°C")
print(f"212°F = {round(fahrenheit_a_celsius(212), 1)}°C")
```

Explicación:

Dato curioso: $-40^{\circ}\text{C} = -40^{\circ}\text{F}$. Es el único punto donde ambas escalas coinciden.

Usar funciones te permite crear una "caja de herramientas" de conversiones reutilizables.

Tema 3: Funciones con Condicionales

Las funciones pueden contener cualquier lógica dentro, incluyendo condicionales (if/elif/else). Esto te permite crear funciones inteligentes que toman decisiones según los datos que reciben.

Ejercicio 3.1 — ¿Es mayor de edad?

Enunciado

Crea una función llamada `es_mayor_edad` que reciba una edad como parámetro. Si la edad es mayor o igual a 18, debe retornar `True`. Si no, debe retornar `False`. Luego usa la función dentro de un `if` para mostrar un mensaje.

Guía paso a paso:

Paso 1: Define la función que retorna `True` o `False` (booleano).

```
def es_mayor_edad(edad):
    if edad >= 18:
        return True
    else:
        return False
```

Esta función retorna un valor booleano (`True` o `False`). Es exactamente el Ejercicio 2 de la actividad práctica de las diapositivas. En PSeInt: Si `edad >= 18` Entonces Devolver Verdadero SiNo Devolver Falso.

Paso 2: Usa la función dentro de un condicional.

```
edad_usuario = int(input("Ingresa tu edad: "))

if es_mayor_edad(edad_usuario):
    print("Eres mayor de edad. Puedes continuar.")
else:
    print("Eres menor de edad.")
```

`es_mayor_edad(edad_usuario)` retorna `True` o `False`, y ese valor se usa directamente en el `if`. Es como preguntar: "¿es mayor de edad? si es verdadero, entonces...".

Paso 3: Versión simplificada (buena práctica).

```
def es_mayor_edad(edad):
    return edad >= 18    # Retorna True o False directamente
```

La expresión `edad >= 18` YA produce `True` o `False` por sí sola. No necesitas el `if/else`. Esta versión es más elegante y profesional.

Mini Reto 3.1

Crea una función `es_par(numero)` que retorne `True` si el número es par, `False` si es impar.

Pista: un número es par si `numero % 2 == 0`.

Prueba: `es_par(4) → True`, `es_par(7) → False`.

Solución sugerida — Ejercicio 3.1

```
def es_mayor_edad(edad):
    return edad >= 18

# Uso
edad = int(input("Ingresa tu edad: "))
if es_mayor_edad(edad):
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")

# Mini reto
def es_par(numero):
    return numero % 2 == 0

print(es_par(4))    # True
print(es_par(7))    # False
```

Explicación:

Las funciones que retornan True/False son muy útiles para validaciones.

Buena práctica: nombra estas funciones con `es_` o `tiene_` para indicar que retornan booleano. `return` con una expresión de comparación es más limpio que `if/else` con `return True/False`.

Ejercicio 3.2 — Calcular descuento

Enunciado

Crea una función llamada `calcular_descuento` que reciba el precio de un producto y el porcentaje de descuento. La función debe calcular el valor del descuento, restárselo al precio y retornar el precio final. Identifica: ¿cuáles son los parámetros? ¿Qué devuelve? ¿Cuáles son los argumentos al invocarla?

Guía paso a paso:

Paso 1: Define la función.

```
def calcular_descuento(precio, porcentaje):
    descuento = precio * porcentaje / 100
    precio_final = precio - descuento
    return precio_final
```

Parámetros: `precio` y `porcentaje` (las variables que recibe). Dentro de la función hacemos el cálculo y retornamos el precio ya con el descuento aplicado.

Paso 2: Invoca la función.

```
final = calcular_descuento(100, 20)
print(f"Precio final: ${final}")    # Precio final: $80.0
```

Argumentos: 100 (precio) y 20 (porcentaje). La función calcula: $100 * 20 / 100 = 20$ de descuento, luego $100 - 20 = 80$. Retorna 80.

Paso 3: Usa la función con datos del usuario.

```
precio = float(input("Precio del producto: $"))
descuento = float(input("Porcentaje de descuento: "))

precio_con_descuento = calcular_descuento(precio, descuento)
print(f"Precio original: ${precio}")
print(f"Descuento: {descuento}%")
print(f"Precio final: ${precio_con_descuento}")
```

Mini Reto 3.2

Agrega una validación: si el porcentaje es menor a 0 o mayor a 100, la función debe retornar el precio original sin descuento.

Pista: usa un if al inicio de la función.

Solución sugerida — Ejercicio 3.2

```
def calcular_descuento(precio, porcentaje):
    if porcentaje < 0 or porcentaje > 100:
        return precio # Porcentaje inválido, retorna sin descuento
    descuento = precio * porcentaje / 100
    return precio - descuento
```

```
# Pruebas
print(calcular_descuento(100, 20)) # 80.0
print(calcular_descuento(50, 10)) # 45.0
print(calcular_descuento(200, 150)) # 200 (porcentaje inválido)
```

Explicación:

Una función puede tener MÚLTIPLES return. El primero que se ejecute termina la función. Validar datos dentro de las funciones es una excelente práctica profesional. Este ejercicio corresponde al Ejercicio 1 de la actividad práctica de las diapositivas.

Ejercicio 3.3 — Calculadora con función

Enunciado

Crea una función llamada calculadora que reciba tres parámetros: num1, num2 y operacion (una letra: "s" para suma, "r" para resta, "m" para multiplicación, "d" para división). La función debe retornar el resultado de la operación correspondiente.

Guía paso a paso:

Paso 1: Define la función con if/elif/else para cada operación.

```
def calculadora(num1, num2, operacion):
    if operacion == "s":
        return num1 + num2
    elif operacion == "r":
        return num1 - num2
    elif operacion == "m":
        return num1 * num2
    elif operacion == "d":
        if num2 != 0:
            return num1 / num2
        else:
            return "Error: división por cero"
    else:
        return "Operación no válida"
```

La función recibe 3 parámetros: dos números y la operación deseada. Según el valor de operacion, realiza un cálculo diferente y lo retorna. Nota la validación para no dividir entre cero. En PSeInt esto se hacía con Segun op Hacer.

Paso 2: Usa la función con datos del usuario.

```
n1 = float(input("Primer número: "))
n2 = float(input("Segundo número: "))
op = input("Operación (s/r/m/d): ")

resultado = calculadora(n1, n2, op)
print(f"Resultado: {resultado}")
```

Comparación PSeInt vs Python:

En PSeInt (Pseudocódigo)	En Python
<pre>Funcion resultado<-calculadora(n1,n2,op) Segun op Hacer Caso "s": resultado = n1+n2 Caso "r": resultado = n1-n2 Fin Segun Fin Funcion</pre>	<pre>def calculadora(n1, n2, op): if op == "s": return n1 + n2 elif op == "r": return n1 - n2 # ... etc.</pre>

Mini Reto 3.3

Agrega la operación de potencia con la letra "p" (num1 ** num2).

Agrega la operación de módulo (residuo) con la letra "o" (num1 % num2).

Solución sugerida — Ejercicio 3.3

```
def calculadora(num1, num2, operacion):
```

```
if operacion == "s":
    return num1 + num2
elif operacion == "r":
    return num1 - num2
elif operacion == "m":
    return num1 * num2
elif operacion == "d":
    if num2 != 0:
        return num1 / num2
    return "Error: división por cero"
elif operacion == "p":
    return num1 ** num2
elif operacion == "o":
    return num1 % num2
return "Operación no válida"

# Pruebas
print(calculadora(10, 3, "s"))    # 13
print(calculadora(10, 3, "d"))    # 3.333...
print(calculadora(2, 8, "p"))     # 256
```

Explicación:

Una función puede tener múltiples return. Solo se ejecuta el primero que se alcance.

Siempre valida la división por cero: es un error muy común.

Este patrón (función con múltiples opciones) es muy usado en programación real.

Tema 4: Funciones de Validación

Las funciones de validación verifican si un dato cumple ciertas condiciones. Retornan True o False y son fundamentales en programación real (validar contraseñas, formularios, datos de entrada, etc.).

Ejercicio 4.1 — Validar contraseña

Enunciado

Crea una función llamada `validar_contraseña` que reciba una contraseña (texto) y verifique tres reglas: 1) Mínimo 8 caracteres, 2) Al menos un número, 3) Al menos una mayúscula. Si cumple las tres, retorna True. Si falla alguna, retorna False.

Guía paso a paso:

Paso 1: Verifica la longitud mínima.

```
def validar_contraseña(clave):  
    # Regla 1: mínimo 8 caracteres  
    if len(clave) < 8:  
        return False
```

Si la contraseña tiene menos de 8 caracteres, inmediatamente retornamos False. La función termina aquí y no evalúa las demás reglas. Esto se llama "retorno temprano" (early return).

Paso 2: Verifica que tenga al menos un número.

```
    # Regla 2: al menos un número  
    tiene_numero = False  
    for caracter in clave:  
        if caracter.isdigit():  
            tiene_numero = True  
            break  
    if not tiene_numero:  
        return False
```

Recorremos cada carácter de la clave. El método `.isdigit()` retorna True si el carácter es un número (0-9). Si encontramos uno, marcamos `tiene_numero = True` y salimos del bucle con `break`. Si al final no encontramos ninguno, retornamos False.

Paso 3: Verifica que tenga al menos una mayúscula.

```
    # Regla 3: al menos una mayúscula  
    tiene_mayuscula = False  
    for caracter in clave:  
        if caracter.isupper():  
            tiene_mayuscula = True  
            break  
    if not tiene_mayuscula:  
        return False
```

`.isupper()` retorna True si el carácter es una letra mayúscula. La lógica es idéntica al paso anterior.

Paso 4: Si pasó todas las validaciones, retorna True.

```
# Si llega aquí, cumple las 3 reglas
return True
```

Paso 5: Usa la función.

```
clave = input("Crea tu contraseña: ")

if validar_contrasena(clave):
    print("Contraseña válida")
else:
    print("Contraseña inválida")
```

Métodos útiles para validar caracteres

.isdigit() → True si es un número ("5".isdigit() = True).
 .isupper() → True si es mayúscula ("A".isupper() = True).
 .islower() → True si es minúscula ("a".islower() = True).
 .isalpha() → True si es una letra ("z".isalpha() = True).

Mini Reto 4.1

Agrega una 4ta regla: la contraseña debe tener al menos una minúscula.

Modifica la función para que en vez de retornar solo True/False, retorne un MENSAJE indicando qué regla falló.

Solución sugerida — Ejercicio 4.1

```
def validar_contrasena(clave):
    if len(clave) < 8:
        return False

    tiene_numero = False
    tiene_mayuscula = False
    for caracter in clave:
        if caracter.isdigit():
            tiene_numero = True
        if caracter.isupper():
            tiene_mayuscula = True

    if not tiene_numero:
        return False
    if not tiene_mayuscula:
        return False

    return True

# Pruebas
print(validar_contrasena("MiClave123")) # True
print(validar_contrasena("corta1A"))    # False (< 8)
```



```
print(validar_contrasena("sinmayusculas1")) # False  
print(validar_contrasena("SINNUMEROS")) # False
```

Explicación:

El retorno temprano (return False apenas falla) hace el código más limpio.

Combinamos bucles (for), condicionales (if) y booleanos dentro de una sola función.

En programación real, validar contraseñas es una tarea muy común y siempre se hace con funciones.

Tema 5: Funciones que Llaman a Otras Funciones

El verdadero poder de las funciones aparece cuando las combinas: una función puede llamar a otras funciones. Esto te permite construir programas organizados por módulos (modularidad), como bloques de LEGO.

Ejercicio 5.1 — Promedio de notas con funciones combinadas

Enunciado

Crea tres funciones: 1) `pedir_nota(numero)` que pida una nota al usuario y la retorne. 2) `calcular_promedio(n1, n2, n3, n4)` que calcule y retorne el promedio. 3) `clasificar_promedio(promedio)` que retorne un mensaje según el promedio (pierde, recuperación o aprueba). Luego crea una función principal que combine las tres.

Guía paso a paso:

Paso 1: Función para pedir una nota.

```
def pedir_nota(numero):  
    nota = float(input(f"Ingresar la nota {numero}: "))  
    return nota
```

Esta función tiene UNA responsabilidad: pedir una nota al usuario y retornarla. El parámetro `numero` sirve para mostrar cuál nota se está pidiendo (1, 2, 3, 4).

Paso 2: Función para calcular el promedio.

```
def calcular_promedio(n1, n2, n3, n4):  
    return (n1 + n2 + n3 + n4) / 4
```

Paso 3: Función para clasificar el promedio.

```
def clasificar_promedio(promedio):  
    if promedio < 3:  
        return "No pasa la materia"  
    elif promedio < 4:  
        return "Va a recuperación"  
    else:  
        return "Aprobó la materia"
```

Paso 4: Función principal que combina las tres.

```
def programa_notas():  
    # Usar pedir_nota para obtener las 4 notas  
    nota1 = pedir_nota(1)  
    nota2 = pedir_nota(2)  
    nota3 = pedir_nota(3)  
    nota4 = pedir_nota(4)  
  
    # Calcular promedio  
    prom = calcular_promedio(nota1, nota2, nota3, nota4)  
    prom = round(prom, 2)
```

```
# Clasificar y mostrar
mensaje = clasificar_promedio(prom)
print(f"Promedio: {prom} - {mensaje}")

# Ejecutar el programa
programa_notas()
```

Cada función hace UNA cosa (principio de responsabilidad única). La función principal las coordina. Esto hace que el código sea fácil de leer, mantener y depurar.

Beneficio de dividir en funciones

Si necesitas cambiar cómo se piden las notas, solo modificas `pedir_nota()`.

Si cambia la fórmula del promedio, solo tocas `calcular_promedio()`.

Si cambian los rangos de clasificación, solo editas `clasificar_promedio()`.

"Divide y vencerás" — la clave de las funciones (como dice la diapositiva).

Mini Reto 5.1

Agrega una función `validar_nota(nota)` que verifique que la nota esté entre 0 y 5.

Modifica `pedir_nota()` para que use `validar_nota()` y siga pidiendo hasta obtener una nota válida.

Solución sugerida — Ejercicio 5.1

```
def validar_nota(nota):
    return 0 <= nota <= 5

def pedir_nota(numero):
    while True:
        nota = float(input(f"Nota {numero}: "))
        if validar_nota(nota):
            return nota
        print("Error: la nota debe estar entre 0 y 5.")

def calcular_promedio(n1, n2, n3, n4):
    return round((n1 + n2 + n3 + n4) / 4, 2)

def clasificar_promedio(promedio):
    if promedio < 3:
        return "No pasa"
    elif promedio < 4:
        return "Recuperación"
    return "Aprobado"

def programa_notas():
    n1 = pedir_nota(1)
    n2 = pedir_nota(2)
    n3 = pedir_nota(3)
```

```
n4 = pedir_nota(4)
prom = calcular_promedio(n1, n2, n3, n4)
print(f"Promedio: {prom} - {clasificar_promedio(prom)}")
```

```
programa_notas()
```

Explicación:

0 <= nota <= 5 es una comparación encadenada de Python (muy elegante).

while True con return dentro crea un bucle que repite hasta obtener un dato válido.

5 funciones pequeñas y claras son mejor que 1 función gigante y confusa.

PARTE 2: EJERCICIOS INDEPENDIENTES

Practica lo aprendido sin guía — Solo enunciado y datos de prueba

Definir e Invocar Funciones

Ejercicio P1 — Despedida personalizada

Enunciado

Crea una función `despedir(nombre)` que imprima "Hasta luego, [nombre]! Fue un gusto." y NO retorne nada. Invócala con 3 nombres.

Ejemplo de uso	Salida esperada
<code>despedir("Ana")</code> <code>despedir("Luis")</code> <code>despedir("Sofía")</code>	Hasta luego, Ana! Fue un gusto. Hasta luego, Luis! Fue un gusto. Hasta luego, Sofía! Fue un gusto.

Pista: Usa `print()` dentro de la función. No necesitas `return`.

Ejercicio P2 — Doble de un número

Enunciado

Crea una función `doble(numero)` que retorne el doble del número recibido. Usa la función para calcular el doble de 7, de 15 y de 100.

Ejemplo de uso	Salida esperada
<code>doble(7)</code> <code>doble(15)</code> <code>doble(100)</code>	14 30 200

Pista: `return numero * 2`

Ejercicio P3 — Elevar al cuadrado

Enunciado

Crea una función `al_cuadrado(n)` que retorne `n` elevado al cuadrado. Luego calcula la suma de los cuadrados de 3 y 4.

Ejemplo de uso	Salida esperada
<code>al_cuadrado(3)</code>	9
<code>al_cuadrado(4)</code>	16
<code>al_cuadrado(3) + al_cuadrado(4)</code>	25

Pista: Usa el operador `**` para potencia.

Funciones con Fórmulas

Ejercicio P4 — Calcular el IMC

Enunciado

Crea una función `calcular_imc(peso, estatura)` que reciba el peso en kg y la estatura en metros, y retorne el Índice de Masa Corporal. Fórmula: $IMC = \text{peso} / \text{estatura}^2$.

Ejemplo de uso	Salida esperada
<code>calcular_imc(70, 1.75)</code>	22.86

Pista: `return round(peso / estatura ** 2, 2)`

Ejercicio P5 — Convertir km/h a m/s

Enunciado

Crea una función `kmh_a_ms(velocidad_kmh)` que convierta una velocidad de km/h a m/s. Fórmula: $m/s = km/h / 3.6$. Prueba con 100 km/h y 36 km/h.

Ejemplo de uso	Salida esperada
<code>kmh_a_ms(100)</code>	27.78
<code>kmh_a_ms(36)</code>	10.0

Pista: `return round(velocidad_kmh / 3.6, 2)`

Ejercicio P6 — Calcular interés simple

Enunciado

Crea una función `interes_simple(capital, tasa, tiempo)` que retorne el interés generado. Fórmula: $I = \text{capital} \times \text{tasa} \times \text{tiempo} / 100$. Prueba: `capital=1000`, `tasa=5%`, `tiempo=3` años.

Ejemplo de uso	Salida esperada
<code>interes_simple(1000, 5, 3)</code>	150.0

*Pista: $\text{return capital} * \text{tasa} * \text{tiempo} / 100$*

Funciones con Condicionales

Ejercicio P7 — Clasificar temperatura

Enunciado

Crea una función `clasificar_temp(grados)` que retorne: "Frío" si < 15 , "Templado" si ≥ 15 y < 25 , "Caliente" si ≥ 25 .

Ejemplo de uso	Salida esperada
<code>clasificar_temp(10)</code>	"Frío"
<code>clasificar_temp(20)</code>	"Templado"
<code>clasificar_temp(35)</code>	"Caliente"

Pista: Usa if/elif/else con return en cada rama.

Ejercicio P8 — El mayor de tres números

Enunciado

Crea una función `mayor_de_tres(a, b, c)` que reciba tres números y retorne el mayor de los tres.

Ejemplo de uso	Salida esperada
<code>mayor_de_tres(5, 12, 8)</code>	12
<code>mayor_de_tres(20, 3, 20)</code>	20

Pista: Puedes usar la función incorporada `max(a, b, c)` o hacerlo con if/elif/else.

Ejercicio P9 — Clasificar nota con letra

Enunciado

Crea una función `nota_letra(nota)` que retorne una letra según la nota numérica: A (≥ 4.5), B (≥ 4.0), C (≥ 3.5), D (≥ 3.0), F (< 3.0).

Ejemplo de uso	Salida esperada
<pre>nota_letra(4.7) nota_letra(3.2) nota_letra(2.5)</pre>	<pre>"A" "D" "F"</pre>

Pista: Usa if/elif/else evaluando de mayor a menor.

Ejercicio P10 — Verificar contraseña simple

Enunciado

Crea una función `verificar_clave(clave)` que retorne `True` si la clave tiene al menos 6 caracteres y contiene al menos un número. `False` en caso contrario.

Ejemplo de uso	Salida esperada
<pre>verificar_clave("hola12") verificar_clave("corto") verificar_clave("sinNumero")</pre>	<pre>True False False</pre>

Pista: Usa `len()` para la longitud y un `for` con `.isdigit()` para buscar números.

Funciones Combinadas y Avanzadas

Ejercicio P11 — Menú de conversiones

Enunciado

Crea 3 funciones: `celsius_a_fahrenheit(c)`, `km_a_millas(km)` y `kg_a_libras(kg)`. Luego crea una función `menu_conversiones()` que pida al usuario qué conversión desea, pida el valor y muestre el resultado usando la función correspondiente.

Ejemplo de uso	Salida esperada
Opción: 1 (Celsius a F) Valor: 100	100°C = 212.0°F

*Pista: Fórmulas: $^{\circ}\text{F} = \text{C} * 9 / 5 + 32$, $\text{millas} = \text{km} * 0.621371$, $\text{libras} = \text{kg} * 2.20462$.*

Ejercicio P12 — Tabla de multiplicar con función

Enunciado

Crea una función `tabla_multiplicar(numero)` que imprima la tabla de multiplicar del 1 al 10 del número dado. Luego pide un número al usuario y muestra su tabla.

Ejemplo de uso	Salida esperada
<code>tabla_multiplicar(5)</code>	5 x 1 = 5 5 x 2 = 10 ... 5 x 10 = 50

Pista: Usa un `for i in range(1, 11)` dentro de la función.

Ejercicio P13 — Contraseña con múltiples mensajes

Enunciado

Crea una función `analizar_contraseña(clave)` que retorne una lista de problemas encontrados. Si la clave es válida (8+ caracteres, tiene número y mayúscula), retorna "Contraseña válida". Si no, retorna los problemas.

Ejemplo de uso	Salida esperada
analizar_contrasena("Ab1") analizar_contrasena("MiClave123")	"Muy corta" "Contraseña válida"

Pista: Puedes acumular mensajes en una variable texto y retornarla al final.

Ejercicio P14 — Calculadora completa

Enunciado

Crea un programa con funciones separadas: sumar(a,b), restar(a,b), multiplicar(a,b), dividir(a,b). Crea una función menu() que muestre opciones, pida datos y llame a la función correcta. Debe permitir hacer múltiples operaciones hasta que el usuario escriba "salir".

Ejemplo de uso	Salida esperada
Op: 1, Nums: 10, 3 Op: 4, Nums: 10, 3	Suma: 13 División: 3.33

Pista: Usa while True con break cuando el usuario elija salir.

Ejercicio P15 — Juego de adivinanza con funciones

Enunciado

Crea funciones: generar_secreto() que retorne un número fijo (ej: 42), evaluar_intento(secreto, intento) que retorne "mayor", "menor" o "correcto", y jugar() que combine todo pidiendo intentos hasta acertar y mostrando cuántos intentos tomó.

Ejemplo de uso	Salida esperada
Intento: 30 → "Es mayor" Intento: 50 → "Es menor" Intento: 42	¡Correcto! Lo lograste en 3 intentos.

Pista: Usa while True dentro de jugar(). Llama a evaluar_intento() en cada vuelta.

¡Felicidades por completar los ejercicios!

"Divide y vencerás" — La clave de las funciones

Las funciones son el primer paso hacia la programación profesional.
Escribe una vez, reutiliza muchas veces. Organiza, simplifica, conquista.